

# get\_wifi

REFERENCE

API consumer

read the reader's saved Wi-Fi profiles and connection status

## Overview

`get_wifi` retrieves the **Wi-Fi profiles saved on the reader** together with their current connection status and configuration. The response is a list ( `wifiProfiles.wifiConfig[]` ) of profiles; each describes the wireless interface, the associated access point (ESSID, security type, connection status), the IPv4/IPv6 addressing, and a `securityDetails` block keyed by the security type ( `WPA2Personal` , `WPA3Personal` , `WPA2Enterprise` , `WPA3Enterprise` ). It is a read-only query on the device-management ([MGMT](#)) endpoint.

The device **masks** stored credentials — passwords, passphrases, and identities are returned as a mask, never the real secret. Use `get_wifi` to verify which SSID a reader is connected to and confirm its addressing before relying on the wireless uplink.

## Details

| Property                | Value                                                                               |
|-------------------------|-------------------------------------------------------------------------------------|
| Interaction Pattern     | Request / Response (app publishes a command; device publishes the correlated reply) |
| State Change            | READ-ONLY                                                                           |
| Required Request Fields | <code>command</code> , <code>requestId</code>                                       |
| Applies To              | <b>Endpoint:</b> Management (MGMT). <b>Reader:</b> RFD40 series, RFD90 series       |
| API Version             | <code>v1.1</code> ( <a href="#">API Version Reporting</a> )                         |
| Related                 | <code>set_wifi</code> , <code>delete_wifi_profile</code> , <code>get_eth</code>     |

## When to Use

Use `get_wifi` when you need to **verify the reader's wireless configuration** — during commissioning, a connectivity check before inventory, or a fleet audit of which SSID each device is on.

- **Confirm the connection:** check `accessPoint.status` (CONNECTED?), `accessPoint.essid` (correct network?), and `accessPoint.isPreferred`.

- **Validate security:** check `accessPoint.securityType` matches the AP; the matching `securityDetails` block shows the (masked) credential/protocol configuration.
- **Validate addressing:** check `ipv4Configuration.ipAddress`, `gateway`, and `enableDhcp` to confirm the reader has a working route to the broker / back-end.

## Topic Mapping

| Message  | Direction    | Topic                                                          | Note                                                                                                |
|----------|--------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Request  | App → Reader | <code>&lt;tenantId&gt;/MGMT/clients/cmnd/&lt;serial&gt;</code> | On the management endpoint.                                                                         |
| Response | Reader → App | <code>&lt;tenantId&gt;/MGMT/clients/resp/&lt;serial&gt;</code> | Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics. |

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

## Key Fields

- `requestId` (request) — your correlation token; make it unique per request. The device echoes it in the response so you can pair reply to request.
- `wifiProfiles.wifiConfig[]` (response) — the array of saved profiles; iterate it to find the connected one.
- `...accessPoint.status` / `essid` (response) — the association state and network name of each profile.
- `...accessPoint.securityType` (response) — selects which `securityDetails` key is populated.
- `...ipv4Configuration.ipAddress` (response) — the DHCP-assigned address the reader uses to reach the broker.

- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

## Request Payload

The request envelope is just `command` and `requestId` — there is no payload wrapper. Use the **Schema** tab for the field reference and the **Example** tab for a ready-to-send, copy-pasteable request.

### Request payload

```
1 {
2   "command" : "get_wifi" ,
3   "requestId" : "abc123"
4 }
```

### Request payload

\* required • — not applicable • Colored badge = JSON type

| # | FIELD            | TYPE   | ENUM / UNITS | EXAMPLE  | DESCRIPTION                                                                                                  |
|---|------------------|--------|--------------|----------|--------------------------------------------------------------------------------------------------------------|
| 1 | command *<br>and | string | —            | get_wifi | Fixed command name. Retrieves the Wi-Fi profiles saved on the device.                                        |
| 2 | requestId *<br>d | string | —            | abc123   | A unique identifier for the request, allowing tracking and debugging. Echoed back unchanged in the response. |

## Response Payload

The response envelope is `command`, `requestId`, `apiVersion`, `response`, and a `wifiProfiles` object whose `wifiConfig` is the **array of profiles**. On the **Schema** tab, expand `wifiProfiles` → `wifiConfig` → `interfaceDetails` (and its `accessPoint`, `ipv4Configuration`, `ipv6Configuration`, `securityDetails` sub-objects) to see every field. Within each profile's `interfaceDetails`, only `interfaceName`, `hostname`, and `macAddress` are required; the interface-level `status` is optional. `securityDetails` carries only the key matching `accessPoint.securityType`, with secrets masked. The **Example** tab shows a WPA2-Personal profile; enterprise variants are in [Examples](#).

### Connection scenario

WPA2 Personal connection ▼

## Response payload

Example: WPA2-Personal

```
1  {
2    "command" : "get_wifi" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "wifiProfiles" : {
6      "wifiConfig" : [
7        {
8          "interfaceDetails" : {
9            "interfaceName" : "wlan0" ,
10           "status" : "" ,
11           "hostname" : "RFD40-211545201D0034" ,
12           "macAddress" : "AC:3F:A4:00:2B:15" ,
13           "accessPoint" : {
14             "essid" : "IOT_TESTAP" ,
15             "status" : "CONNECTED" ,
16             "securityType" : "WPA2Personal" ,
17             "isPreferred" : false ,
18             "autoConn" : false
19           } ,
20           "ipv4Configuration" : {
21             "ipAddress" : "192.168.0.110" ,
22             "subnetMask" : "255.255.255.0" ,
23             "gateway" : "192.168.0.1" ,
24             "enableDhcp" : true ,
25             "dnsServer" : "192.168.0.1" ,
26             "domainName" : "zebra.com"
27           } ,
28           "securityDetails" : {
29             "WPA2Personal" : {
30               "password" : "XXXXXXXX"
31             }
32           }
33         }
34       ]
35     } ,
36     "response" : {
37       "code" : 0 ,
38       "description" : "Success"
39     }
40   }
41 }
```

## Response payload

\* required   •   — not applicable   •   Colored badge = JSON type

| #  | FIELD                | TYPE   | ENUM / UNITS               | EXAMP |
|----|----------------------|--------|----------------------------|-------|
| 1  | command *            | string | —                          | get_v |
| 2  | requestId *          | string | —                          | abc12 |
| 3  | apiVersion *         | enum   | V1.0 V1.1 V1.2 V1.21 V1.22 | V1.1  |
| 4  | wifiProfiles         | object | —                          | —     |
| 5  | wifiConfig           | array  | —                          | —     |
| 6  | interfaceDe<br>tails | object | —                          | —     |
| 7  | interfa<br>ceName *  | string | —                          | wlan0 |
| 8  | status               | enum   | Enabled Disabled           | Enabl |
| 9  | hostnam<br>e *       | string | —                          | RFD40 |
| 10 | macAddr<br>ess *     | string | —                          | E0-D0 |

| #  | FIELD             | TYPE    | ENUM / UNITS                                               | EXAMP |
|----|-------------------|---------|------------------------------------------------------------|-------|
| 11 | accessPoint       | object  | —                                                          | —     |
| 12 | essid *           | string  | —                                                          | Zwire |
| 13 | status *          | enum    | CONNECTED DISCONNECTED                                     | CONN  |
| 14 | securityType *    | enum    | WPA2Personal WPA3Personal<br>WPA2Enterprise WPA3Enterprise | WPA2I |
| 15 | isPreferred *     | boolean | —                                                          | false |
| 16 | autoConn          | boolean | —                                                          | false |
| 17 | ipv4Configuration | object  | —                                                          | —     |
| 18 | ipAddress         | string  | ipv4                                                       | 192.1 |
| 19 | subnetMask        | string  | ipv4                                                       | 255.2 |
| 20 | gateway           | string  | ipv4                                                       | 192.1 |
| 21 | enableDhcp        | boolean | —                                                          | true  |
| 22 | dnsServer         | string  | ipv4                                                       | 192.1 |
| 23 | domainName        | string  | —                                                          | test. |
| 24 | ipv6Configuration | object  | —                                                          | —     |
| 25 | ipAddress         | string  | ipv6                                                       | fe80: |

| #  | FIELD           | TYPE    | ENUM / UNITS  | EXAMP |
|----|-----------------|---------|---------------|-------|
| 26 | prefix          | integer | —             | 64    |
| 27 | gateway         | string  | ipv6          | 2a02  |
| 28 | enableAuto      | boolean | —             | true  |
| 29 | dnsServer       | string  | ipv6          | 2001: |
| 30 | domainName      | string  | —             | test. |
| 31 | securityDetails | object  | —             | —     |
| 32 | WPA2Personal    | object  | —             | —     |
| 33 | password        | string  | password      | test0 |
| 34 | WPA3Personal    | object  | —             | —     |
| 35 | password        | string  | password      | test0 |
| 36 | WPA2Enterprise  | object  | —             | —     |
| 37 | authentication  | enum    | tls ttls peap | tls   |

| #  | FIELD               | TYPE   | ENUM / UNITS                                     | EXAMP |
|----|---------------------|--------|--------------------------------------------------|-------|
| 38 | innerAuthentication | enum   | none mschapv2                                    | mscha |
| 39 | identity            | string | —                                                | test  |
| 40 | anonymousIdentity   | string | —                                                | test  |
| 41 | password            | string | password                                         | test  |
| 42 | phrase              | string | —                                                | test  |
| 43 | certificate         | array  | —                                                | —     |
| 44 | key                 | enum   | ca_cert client_key client_cert cert_key_password | ca_ce |
| 45 | value               | string | —                                                | wifi_ |

| #  | FIELD                                       | TYPE   | ENUM / UNITS         | EXAMP |
|----|---------------------------------------------|--------|----------------------|-------|
| 46 | pro<br>toc<br>ol                            | enum   | WPA2_Enterprise_CCMP | WPA2_ |
| 47 | WPA3En<br>terpri<br>se                      | object | —                    | —     |
| 48 | aut<br>hen<br>tic<br>ati<br>on              | enum   | tls ttls peap        | tls   |
| 49 | inn<br>erA<br>uth<br>ent<br>ica<br>tio<br>n | enum   | none mschapv2        | mscha |
| 50 | ide<br>nti<br>ty                            | string | —                    | test  |
| 51 | ano<br>nym<br>ous<br>Ide<br>nti<br>ty       | string | —                    | test  |
| 52 | pas<br>swo<br>rd                            | string | password             | test  |
| 53 | pas<br>sph<br>ras<br>e                      | string | —                    | test  |

| #  | FIELD                                        | TYPE    | ENUM / UNITS                                                                                                                                           | EXAMP |
|----|----------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 54 | cert<br>t<br>i<br>f<br>i<br>c<br>a<br>t<br>e | array   | —                                                                                                                                                      | —     |
| 55 | k<br>e<br>y                                  | enum    | ca_cert client_key<br>client_cert cert_key_password                                                                                                    | ca_ce |
| 56 | v<br>a<br>l<br>u<br>e                        | string  | —                                                                                                                                                      | wpa3_ |
| 57 | pro<br>to<br>c<br>o<br>l                     | enum    | WPA3_Enterprise_CCMP<br>WPA3_Enterprise_CCMP_256<br>WPA3_Enterprise_GCMP_128<br>WPA3_Enterprise_GCMP_256_SHA256<br>WPA3_Enterprise_GCMP_256_SUITEB_192 | WPA3_ |
| 58 | response *                                   | object  | —                                                                                                                                                      | —     |
| 59 | code *                                       | integer | 0–101                                                                                                                                                  | 0     |
| 60 | description *                                | string  | —                                                                                                                                                      | Succ  |

## Response Semantics

- **Credentials are masked.** The device returns stored passwords, passphrases, and identities as a mask (never the real secret). Treat every credential field in the response as opaque — you cannot read back the actual value with `get_wifi`.
- **Security type selects the credential block.** `securityDetails` contains only the one key that matches `accessPoint.securityType` — `WPA2Personal` / `WPA3Personal` (a single `password`) or `WPA2Enterprise` / `WPA3Enterprise` (EAP `authentication`, optional `innerAuthentication` / `identity` / `anonymousIdentity`, a `certificate` array, and a `protocol`).
- `ipv6Configuration` is schema-defined but not returned by the current firmware.

- **Wi-Fi-disabled response shape.** When Wi-Fi is disabled, the contract's own example returns the profile under a **top-level** `wifiConfig` object (rather than `wifiProfiles.wifiConfig[]`), carrying only `interfaceName`, `status` (`Disabled`), `hostname` and `macAddress`. The response schema does not declare a top-level `wifiConfig`, so parse both shapes; the connected shape documented here is `wifiProfiles.wifiConfig[]`.

## Response Codes

`get_wifi` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

| Code | Description                                                                                                                                                                                                      |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0    | Success — Wi-Fi profiles retrieved and returned.                                                                                                                                                                 |
| 3    | Not able to retrieve information ( <code>IOT_ERROR_INFO_NOT_AVAILABLE</code> ) — the device could not gather the Wi-Fi configuration at this time. Retry after a short delay; if it persists, reboot the device. |
| 101  | Default error — generic catch-all.                                                                                                                                                                               |

For the full code list see the [Response Codes](#). (`Code 20`, `Wifi is not supported`, applies to the Wi-Fi **setters** `set_wifi` / `set_config`, not to `get_wifi`.)

## Examples

`get_wifi` is a single request/response exchange. Publish the standard `{command, requestId}` request shown under [Request Payload](#); the device replies on the response topic with the correlated reply — the `requestId` you send ( `abc123` ) is echoed back so you can pair reply to request. The **Connection scenario** dropdown under [Response Payload](#) shows each contract example:

- **WPA2 Personal connection** — a single masked `password`.
- **WPA2 Enterprise connection** — an EAP `authentication` method, a `certificate` array of `{key, value}` references and a `protocol`, with credentials masked.
- **WPA3 Enterprise connection** — additionally `innerAuthentication`, `identity` and `anonymousIdentity`, and a WPA3 `protocol` suite.
- **WiFi interface disabled** — the minimal reply for a disabled interface.

## Notes and Caveats

`get_wifi` is a read-only getter — safe to poll during commissioning or troubleshooting. Configure profiles with `set_wifi` / `delete_wifi_profile`; for the concept-level view, see [Network Configuration](#). Enterprise profiles depend on installed certificates — see [TLS and Authentication](#).

**Related:**  [Network Configuration](#) ·  `get_eth` ·  Network and Connectivity ·  [Response Codes](#)

get\_wifi

---

2 of 4